Threat-Modelling WebRTC-Based IP and Metadata Leakage on University BYOD Networks: A Campus-Specific Countermeasure Framework with Emulated and On-Device Evaluation

Shubhi Singh
School of Computer Science and Engineering (SCOPE)
Vellore Institute of Technology
Vellore, India

Jilagam Yatin
School of Computer Science and Engineering (SCOPE)
Vellore Institute of Technology
Vellore, India

Dr. Kopperundevi
School of Computer Science and Engineering (SCOPE)
Vellore Institute of Technology
Vellore, India

***Abstract*—WebRTC lets browsers exchange media and data directly, and to do so it gathers and shares the network addresses of the device. University bring-your-own-device (BYOD) wireless networks place large numbers of unmanaged devices on shared eduroam segments, and the operator's usual control, Wi-Fi client isolation, has published bypasses. This paper presents a threat model for that setting. It defines a reference campus architecture, five leakage classes, and four adversaries, and builds attack trees for a lecture hall, a hostel, and a library; their twenty attack steps are mapped to eleven countermeasures ranked by deployment burden, enforceability, and coverage. An emulation of 560 browser-to-browser sessions with Chromium, Firefox, and WebKit under ten network configurations tests the main recommendations. With link-layer isolation alone, browsers that advertised raw addresses still reached each other through the gateway in 16 of 16 sessions; a gateway filter removed that path. Filtering multicast DNS moved default browsers to a relay only on IPv4 behind NAT. With global IPv6 on the segment, every browser disclosed its IPv6 address (168 of 168 sessions) and connected directly over IPv6 despite the filter. On real hardware, 60 sessions of Microsoft Edge on a laptop's Wi-Fi interface reproduced the emulated NAT result. Safari on an iPhone 16 Pro Max (iOS 26.0) hid its local addresses but disclosed public IPv4 and global IPv6 addresses through STUN in both sessions run. The emulation uses a Linux bridge, not Wi-Fi hardware, the device checks are small, and the ranking remains a qualitative judgement.**

***Index Terms*—attack trees, BYOD, client isolation, eduroam, ICE, IP address leakage, threat modelling, WebRTC, wireless network security**

I. Introduction

Web Real-Time Communication (WebRTC) is the browser facility behind web-based video calling, screen sharing, and peer-to-peer data transfer. To connect two browsers directly, its Interactive Connectivity Establishment (ICE) procedure gathers the addresses at which each device may be reachable and shares them with the peer [1]. A script running in a web page can read the same addresses. The result is that a page, or the other party in a call, may learn an address the user did not intend to reveal [2], [3].

The problem is old [4], [5] and only partly fixed. A cross-platform measurement released as a preprint in October 2025 found that desktop Chromium browsers now replace the local address with a multicast DNS (mDNS) name. Chrome and Firefox on Android, by contrast, still disclosed the local address on Wi-Fi and a carrier-grade NAT address on a mobile network [6]. A second study found that on networks without address translation, public addresses leak during calls even when a VPN or proxy is in use, and reported that the eduroam networks of two university campuses were such networks [7].

A university wireless network differs from the home and enterprise networks that most of this literature assumes. Under eduroam, personally owned devices authenticate with IEEE 802.1X through a federated RADIUS hierarchy, and some belong to visitors from other institutions [8]. The operator does not own these devices and cannot require software on them [9]. The same access points commonly serve an authenticated network and an open guest network at once [10].

Two recent results raise the stakes. In March 2026, Sansec reported a payment skimmer that receives its code and sends stolen data over a WebRTC data channel, a path that content security policy and HTTP inspection do not cover [11]. Meanwhile, Wi-Fi client isolation, the control an operator would normally rely on to keep devices on one segment apart, has been bypassed in practice. Schepers *et al.* did so in 2023 by manipulating transmit queues [12], [13], and in 2026 Zhou *et al.* reported attacks that worked on every router and network they tested, two university networks among them [10].

These strands of work have not been connected. The leakage studies stop at the browser, the isolation studies do not ask which application behaviour helps an attacker pick a target, and campus BYOD work stays at the level of policy and general risk [14]. A campus operator who wants to know what the leakage means on a client segment, and which measures to apply first, has no single source to consult.

This paper builds a threat model for that setting by synthesising published work, and then tests the model's central

recommendations in an emulated network and on two physical devices. Its contributions are:

- a reference model of a campus wireless network, with five leakage classes and four adversaries, each tied to published evidence (Section V);
- attack trees for three campus scenarios, a lecture hall, a hostel, and a library, decomposed into twenty attack steps (Section VI);
- eleven countermeasures mapped to those steps and ranked by an explicit rule, with client isolation identified as insufficient alone (Section VII);
- an emulated evaluation of 560 browser-to-browser sessions under ten network configurations, measuring what isolation, gateway filtering, multicast filtering, address translation, and IPv6 each change; the disclosure result is also checked on a physical Wi-Fi interface and on an iPhone (Section VIII).

The evaluation is an emulation with desktop browsers plus small checks on real hardware, not a measurement on a campus network. Section X states what that means for the conclusions.

II. Background

A. ICE Candidates and Address Handling

An ICE agent gathers host candidates from local interfaces, server-reflexive candidates from a STUN server, and relayed candidates from a TURN server [1], [15]. A host candidate may be a private address, an address from the shared space used behind carrier-grade NAT [16], or, on a network without translation, a public address.

RFC 8828 defines four address-handling modes [2]. Mode 1 enumerates all interfaces and requires user consent. Mode 2, the recommended default without consent, uses the default route and still exposes the private addresses associated with it as host candidates. Mode 3 withholds private addresses. Mode 4 forces traffic through the proxy used for HTTP. The same document names VPN exposure as its main concern.

Browsers additionally hide host addresses behind randomly generated mDNS names [17]. The specification for this is an expired Internet-Draft. It notes two side effects. On large networks mDNS may be disabled or limited in scope, so connections fall back to hairpinning or a relay; and a malicious endpoint on the local network can still record which peers communicate.

B. eduroam

eduroam is a roaming service for research and education. The visited institution's access point acts as an 802.1X authenticator and forwards the authentication through a RADIUS proxy hierarchy to the user's home institution [8]. Credentials are visible only to the home institution. The visited institution sees the realm of an outer identity that is normally anonymous, and the device's MAC address. RFC 7593 notes that separating guest traffic from local users with VLANs is possible and widely used. It sets no requirement on address translation or on filtering of visitor traffic; those are local decisions.

TABLE I
Related Work and What It Leaves Open

Cluster	What it establishes	What it leaves open
Leakage measurement	Which browsers disclose which addresses [4], [6], [7]; endpoint fixes [5], [7]; handling modes [2]	Fixes act on the device; no network-topology analysis
WebRTC as attack surface	Peer address exposure [3], [19]; data-channel skimmer [11]	Generic web or enterprise setting
Isolation bypass	Isolation can be defeated, also on university networks [10], [12]	No link to application-layer leakage
Campus BYOD	Risk assessment [14]; architecture [8]; no software on clients [9]	Not tied to a specific protocol behaviour
This paper	Campus attack trees joining leakage and isolation bypass; operator ranking; emulated test	No Wi-Fi hardware or Android

C. Wi-Fi Client Isolation

Client isolation blocks direct communication between stations on the same wireless network and was introduced against attacks such as ARP spoofing [18]. It is not standardised. Schepers *et al.* bypassed it by manipulating the frames an access point queues for a sleeping client [12], [13]. Zhou *et al.* analysed isolation at the encryption, switching, and routing layers and found every tested router and network vulnerable to at least one attack [10]. Their techniques include injecting packets through the gateway at the network layer, abusing the shared group key, and stealing the victim's switch port by connecting with its MAC address. The attacks apply to WPA2 and WPA3 in enterprise as well as personal mode.

III. Related Work

We searched Google Scholar with seven query formulations covering WebRTC leakage, WebRTC threat models in enterprise and BYOD settings, client-isolation bypass, campus BYOD security, and threat modelling of campus networks, and retained 41 sources after screening. Table I summarises the clusters that bear on this paper.

Leakage measurement. Koysha *et al.* tested Chrome, Brave, Firefox, and Tor on Windows, Linux, and Android, with a first phase on a university Wi-Fi network, and outlined a threat model for what they call semi-trusted BYOD environments [6]. Their model classifies networks by trust level and stays at the browser; it does not decompose a campus network or rank operator-side controls. Nibert *et al.* analysed desktop browsers and confined Firefox in a container as the fix [7]. Fakis *et al.* used a browser extension and an HTTP gateway [5]. None of these can be imposed on a visitor's phone.

WebRTC as an attack surface. RFC 8826 records that ICE reveals a party's address to the other party [3]. Tang *et al.* show that peer-assisted video delivery exposes viewers' addresses to other viewers [19]. The Sansec report shows a WebRTC data channel used by criminals as an egress channel [11].

Isolation bypass. The work in [10], [12] establishes that devices on a network configured for isolation can still reach or intercept each other. It treats the choice of victim as given.

Position of this paper. The closest work is [6], which measures on a campus network, and [7], which observes the eduroam addressing issue. Neither builds scenario-level attack trees or an operator-facing ranking. This paper contributes that combination, and an emulated test of the operator-side controls that neither work includes.

IV. Method

The work is a structured synthesis in four steps, followed by an emulated evaluation.

Step 1: evidence extraction. From each anchor source we recorded what was observed, on which platforms, and under which conditions, and separated observations from the authors' interpretation. Only observations are used as evidence.

Step 2: reference model. We defined a campus wireless architecture from RFC 7593 and from the university deployment described in [10]. Every property that comes from neither source is listed as an assumption.

Step 3: attack trees. For each scenario we wrote an attack tree in the sense of Schneier [20]: a root goal, sub-goals that must all be met, and leaf steps that are alternatives. A leaf is included only if a cited source demonstrates the step, or if it is marked as an assumption.

Step 4: countermeasure ranking. Each countermeasure is rated on three ordinal attributes.

- *Burden* is the larger of deployment effort and collateral impact. It is *low* for a configuration change on existing equipment, *medium* for a feature the equipment may not support or one that needs new operational process, and *high* for a redesign or a measure that breaks legitimate use.
- *Enforcement* is *network* if the operator can apply the measure to every device, *managed* if it reaches only university-owned devices, and *voluntary* if it depends on each user.
- *Coverage* is the number of attack steps in Table IV that the measure blocks or exposes.

Countermeasures are sorted by burden, lowest first; then network-enforced before the others; then by coverage, highest first. The ratings are the authors' judgement from the cited sources. The rule deliberately places burden first, because the intended reader is an IT department with limited budget and no control over endpoints.

Step 5: emulated evaluation. The recommendations that can be tested without a campus network were tested with real browsers on an emulated client segment. Section VIII gives the setup and results.

V. Campus Threat Model

A. Reference Architecture

Fig. 1 shows the reference network. Each access point broadcasts three networks, as in the university deployment tested in [10]: eduroam with WPA2-Enterprise, a staff network with WPA2-Enterprise, and an open guest network behind a captive portal. Access points connect through a distribution switch to a gateway. A local RADIUS server authenticates local users and proxies visitors' requests to the federation [8].

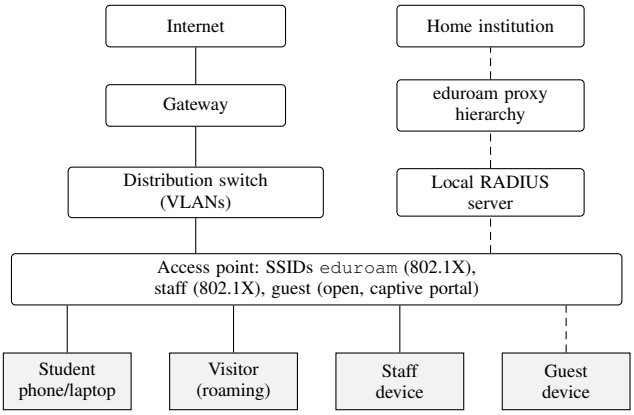


Fig. 1. Reference campus wireless architecture. Solid lines carry user traffic; dashed lines carry authentication, except the guest device, which is not authenticated by 802.1X.

The operator controls the access points, switches, gateway, and local RADIUS server. It controls no student, visitor, or guest device.

Assumptions. The following are not taken from a source and may not hold at a given institution: (A1) clients on one SSID in one building share a VLAN; (A2) client isolation is enabled within each SSID; (A3) address leases in residential buildings last for days; (A4) some personal devices expose services such as file sharing on the local segment.

B. Assets

The assets are a device's local or public address, the link between an address and a person, the device's reachability from other devices, the confidentiality of its traffic, and the data a user enters in a web page.

C. Leakage Classes

Table II lists the five leakage classes used in the rest of the paper, with the evidence for each. The classes describe what an attacker learns or gains; they do not depend on the campus setting.

Two limits of the evidence matter. First, the measurements in [6] did not cover VPN use, Safari, iOS, or IPv6-only networks, and are not peer reviewed. The claim that a VPN does not hide the address rests on [7] and on [4], not on [6]. Second, L5 comes from a vendor report about one campaign against an online shop; it shows the channel is usable, not how common it is.

D. Adversaries

- *Remote origin (R).* Controls a web page or an embedded script that a campus user loads. Has no presence on the campus network.

TABLE II
LEAKAGE CLASSES AND THEIR EVIDENCE

	Class	Published evidence
L1	Local address to a web page	Chrome and Firefox on Android 14 disclosed the Wi-Fi address; both disclosed a carrier-grade NAT address on 4G [6]
L2	Session-stable identifier	Desktop Chromium and Brave emit an mDNS name that is stable within a session [6]
L3	Public address despite VPN or proxy	Observed on desktop browsers on networks without NAT; two campus eduroam networks had none [7]
L4	Address to the other peer	Inherent to direct connectivity [3]; viewers exposed to each other in peer-assisted video [19]
L5	Covert data channel	Skimmer code and stolen data carried over a data channel outside content security policy and HTTP inspection [11]

- *Authenticated insider (I).* A student, staff member, or roaming visitor with valid credentials and a device on the same network as the victim.
- *Guest (G).* A person within radio range who can join the open guest network but holds no credentials.
- *Compromised page (P).* A legitimate site whose pages carry injected code, as in [11].

An adversary who controls the infrastructure or the home institution is out of scope, as is an attack on the 802.1X authentication itself.

E. Why the Campus Setting Changes the Risk

Three properties of the reference network change what the leakage is worth to an attacker.

Density. A device address is of little use among a handful of devices at home. In a lecture hall, hundreds of devices share an access point, and an insider who wants to reach one particular person's device must first find it. A leaked address, obtained through a link the victim opens, does that.

Persistence. In a hostel the same device stays on the same network for months. Identifiers that are stable for a session or a lease can be collected repeatedly and joined over time.

Mixed trust on shared hardware. The guest and authenticated networks share access points and a distribution switch. The isolation bypass in [10] crosses exactly that boundary.

VI. ATTACK TREES FOR THREE CAMPUS SCENARIOS

Table III gives the scenarios. In each tree the three sub-goals must all be achieved, and the leaves under a sub-goal are alternatives. Table IV lists every leaf with its source.

A. Lecture Hall

The attacker is a person in the room who wants to reach a specific classmate's device (Fig. 2). The first sub-goal is to learn which of several hundred addresses belongs to the victim. A link posted to a class group suffices: the page gathers candidates without a prompt, and on Android Chrome or Firefox the candidate is the Wi-Fi address (H1). A browser-based call or file-transfer session between the two has the same

TABLE III
SCENARIOS

Scenario	Defining property	Adversary	Main leakage
Lecture hall	Many devices on one access point for a short period	I	L1, L2, L4
Hostel	Same devices on the same segment for months	R, I	L1, L2, L3
Library	Guest and authenticated users on shared access points	G	L1 (supporting)

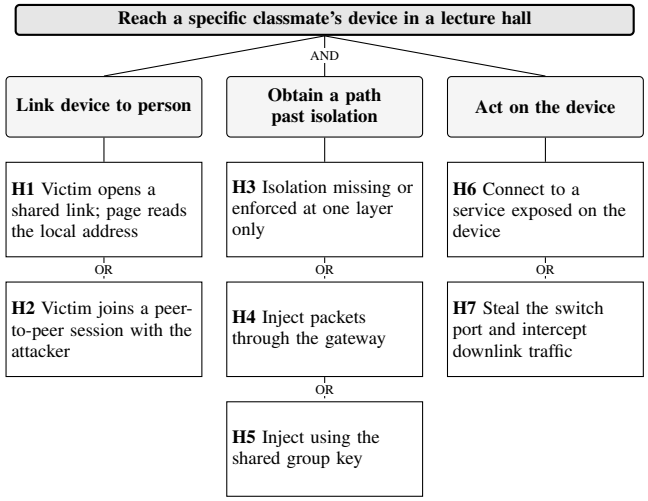


Fig. 2. Attack tree for the lecture hall.

effect (H2). The second sub-goal is a path to that address. It exists if isolation is absent or enforced at one layer only (H3), or it can be created by injecting through the gateway (H4) or with the group key (H5). The third sub-goal is to use the path, against a service the device exposes (H6, assumption A4) or by intercepting the victim's downlink traffic through port stealing (H7).

WebRTC contributes only the first sub-goal. Without it, the insider can still attack devices at random, but cannot easily aim at one person.

B. Hostel

The attacker wants to follow or repeatedly reach one resident's device over weeks (Fig. 3). A durable identifier can come from a call that discloses the public address even though the resident uses a VPN, on a segment without address translation (D1). It can also come from a page the resident visits repeatedly, which records the local address or mDNS name each time (D2). The identifier must then be joined across sessions, which works if the address is stable over a long lease (D3, assumption A3) or if the page combines it with browser metadata such as platform, time zone, and screen properties (D4). The attacker then uses the linkage, locally through an isolation bypass (D5) or remotely by geolocating or flooding a public address (D6).

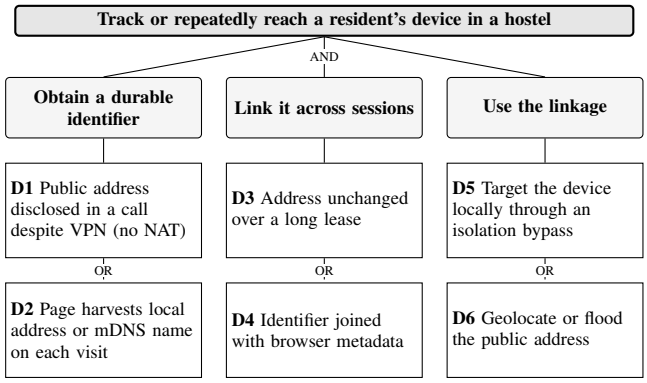


Fig. 3. Attack tree for the hostel.

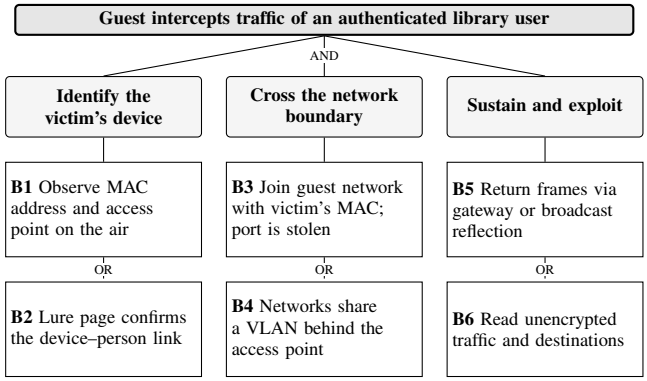


Fig. 4. Attack tree for the library.

Here the remote origin is the main adversary, and the hostel matters because the network is the resident's home network without the address translation a home router provides.

C. Library

The attacker holds no credentials and joins the open guest network (Fig. 4). This tree follows the university experiment in [10]. The attacker identifies the victim's device by observing its MAC address and access point on the air (B1); a lure page that reads the victim's address can confirm which device belongs to which person (B2). The attacker then connects to the guest network using the victim's MAC address, so that the distribution switch redirects the victim's downlink traffic (B3), which is possible when the networks are not separated behind the access point (B4). Finally the attacker returns frames to the victim through the gateway or by broadcast reflection to stay unnoticed (B5) and reads what is not encrypted end to end, together with destination addresses (B6).

WebRTC leakage is not required in this tree. It appears only as B2, where it lowers the effort of choosing a victim. We include the scenario because it is the one in which the cheapest standard control fails most clearly.

D. Cross-Cutting Step: Covert Data Channel

Step X1 is independent of topology. A compromised page (adversary P) opens a data channel to a fixed address and moves code or data over DTLS on UDP [11]. For the campus this has two consequences: users' browsers can be made to exfiltrate what they type, and monitoring that inspects only HTTP does not see it. The reported skimmer sent its traffic to UDP port 3479 [11]; nothing prevents a variant from using a port that is commonly allowed.

VII. Countermeasure Framework

Table V lists eleven countermeasures in ranked order under the rule of Section IV. Coverage counts the steps of Table IV in whose last column the measure appears.

A. Reading the Ranking

First tier, ranks 1 to 5. These need no new equipment. C1 and C2 are the measures that the isolation literature itself recommends [10]. Placing untrusted networks in separate VLANs removes the cross-network port stealing of step B3, and network-layer filtering at the gateway removes the injection of steps H4 and B5. C3 stays in the tier because it costs nothing where it is already supported, but it addresses a single step and is the one control with published, working bypasses. C4 has the highest coverage in the tier and the weakest enforcement: it touches every step in which the victim's browser discloses something, and it helps only the users who follow it.

Second tier, ranks 6 to 8. These depend on features that the installed equipment may lack. C6 does not block anything; it is the only measure in the table that gives visibility into the covert channel of step X1 and into direct client-to-client traffic that should not exist when isolation holds.

Third tier, ranks 9 to 11. C9 has the highest coverage of all eleven measures and is ranked ninth, because the rule places burden first. An operator planning a network refresh should read it as the long-term target and the first tier as what to do meanwhile. C11 ranks last: it addresses one step, partially, and blocks ordinary conferencing traffic.

B. What Is Not Enough Alone

Three common recommendations fail as single measures under this model.

Client isolation. It blocks step H3 and none of H4, H5, H7, B3, or B5. An operator who has enabled isolation has closed the direct path and left the indirect ones.

"Use a VPN." On a client network without address translation, a public address can be disclosed during a call although a VPN is active [7]. The advice is still useful against other threats; it does not close L3.

Relying on browser fixes. The mDNS replacement removes the raw address on desktop Chromium and leaves a session-stable name (L2), and the tested Android browsers still disclosed the address [6]. The operator cannot choose the browser on a personal device.

C. eduroam Compatibility

None of C1 to C3 or C6 to C11 changes the authentication exchange, so none requires cooperation from a visitor's home institution; all act on local traffic after authentication, which

TABLE IV
ATTACK STEPS, THEIR BASIS, AND THE COUNTERMEASURES THAT ADDRESS THEM

Step	Description	Basis	Countermeasures
H1	Page opened by the victim reads the local address without a prompt	Measured on Android Chrome and Firefox [6]	C4, C5
H2	Peer-to-peer session discloses the victim's address to the attacker	[3], [19]	C4
H3	Direct client-to-client traffic because isolation is absent or partial	[10]	C3, C6, C9
H4	Packets injected to the victim through the gateway at the network layer	[10]	C1, C6, C9
H5	Packets injected to the victim in frames protected by the group key	[10]	C8, C9
H6	Connection to a service the device exposes on the segment	Assumption A4; cf. [7]	via H3–H5
H7	Downlink traffic intercepted by port stealing on the same network	[10]	C7, C9
D1	Public address disclosed during a call although a VPN or proxy is used	Desktop browsers, non-NAT network [7]	C4, C10
D2	Local address or mDNS name harvested by a page on each visit	[6]	C4, C5
D3	Address remains the same across sessions	Assumption A3	C10
D4	Identifier combined with browser metadata to link sessions	[6]	C4
D5	Device targeted from the same segment through an isolation bypass	[10]	C1, C9
D6	Public address geolocated or flooded from outside	Stated as a consequence in [7]	C10
B1	Victim's MAC address and access point observed on the air	[10]	none
B2	Lure page confirms which device belongs to the victim	[6]	C4, C5
B3	Guest joins with the victim's MAC address; downlink traffic is redirected	University test in [10]	C2, C7, C9
B4	Guest and authenticated networks are not separated behind the access point	[10]	C2
B5	Intercepted frames returned through the gateway or by broadcast reflection	[10]	C1, C9
B6	Unencrypted content and destination addresses read	[10]	via B3–B5
X1	Compromised page moves code or data over a WebRTC data channel	One reported campaign [11]	C6, C11

RFC 7593 leaves to the visited institution [8]. C4 and C5 do not reach roaming visitors at all. The anonymity of the outer identity means the visited institution usually knows a visitor's device only by realm and MAC address, which limits how far any detection under C6 can be attributed to a person.

VIII. EMULATED EVALUATION

A. Questions

Four claims of the framework can be tested without a campus network. (Q1) Does link-layer isolation alone stop two browsers on one segment from connecting to each other, and does the gateway rule C1 close what isolation leaves open? (Q2) Does filtering multicast suppress direct connections between browsers that hide their addresses behind mDNS names, as Section II suggests it might? (Q3) What does the other peer learn under translated, public, and dual-stack addressing? (Q4) Do sessions still work when the controls are on?

B. Setup

The testbed runs on one Linux host (kernel 6.18) with network namespaces. Two client namespaces attach to ports of a bridge that stands for the client segment. A gateway namespace routes to an outside host that runs coturn 4.5.2 as STUN and TURN server. The link between gateway and outside host carries 20 ms of added delay in each direction, so that a relayed path is slower than a local one, as it is in practice.

Addressing. In the *NAT* mode clients hold private IPv4 addresses and the gateway translates; IPv6 is disabled. In the *public* mode clients hold routable IPv4 addresses and nothing is translated, the situation reported for two campus eduroam networks in [7]. In the *dual* mode clients have the NAT mode's IPv4 and, in addition, global IPv6 addresses that are routed without translation.

Controls. Isolation is bridge port isolation on both client ports, which blocks every frame between them, multicast included. *mDNS filtering* drops UDP port 5353 arriving from a client port, for IPv4 and IPv6. The gateway either answers ARP on behalf of other clients and forwards between them, as private-VLAN deployments do, or drops traffic that would leave by the client interface it arrived on, which is countermeasure C1.

Browsers. Chromium 153.0.8010.12, Firefox 155.0, and WebKit 26.6, the builds shipped with Playwright, run headless. Seven browser configurations are used. Each of the three browsers runs in its default configuration. Chromium and Firefox also run with microphone permission granted and an audio track added, as in a call, and in a *raw-host* configuration in which a browser flag or preference switches mDNS obfuscation off. The raw-host configuration stands in for browsers that advertise the interface address, as [6] reports for Android. It is not an Android browser, and the WebKit build is not Safari.

Procedure. A session starts a fresh browser in each client, opens a data channel, and exchanges offer and answer through the test harness, so signalling does not cross the emulated network. The harness waits up to 12 s for the connection and confirms it with a message echoed over the channel. It records the candidates, the selected candidate pair, and the packets from client 1 to client 2's addresses at client 1's bridge port. A session is classed as *relay* if a selected candidate is relayed, *direct* if frames were addressed to the peer's MAC address, and *via gateway* if they were addressed to the gateway's MAC address. Ten configurations, seven browser configurations, and

TABLE V
RANKED COUNTERMEASURES FOR A CAMPUS OPERATOR

Rank	ID	Countermeasure	Burden	Enforcement	Coverage	Residual weakness
1	C1	Gateway rule that refuses to forward between client addresses, with IP source validation	Low	Network	3	Closed the gateway path in emulation (Section VIII); same-segment sessions then need a relay, and some still failed; equipment support not surveyed
2	C2	Separate VLAN for each SSID and role; guest network never bridged with authenticated networks	Low	Network	2	Attacks inside one VLAN remain; effect shown on one access-point model in [10]
3	C3	Client isolation enabled on every network	Low	Network	1	**Bypassable [10], [12]; insufficient alone.** Left the gateway path open in emulation
4	C4	User guidance: browser choice and settings, limits of a VPN, caution with shared links	Low	Voluntary	6	Cannot be enforced; Android defaults are weaker than desktop [6]
5	C5	Browser address-handling policy (Mode 3 or 4 of [2]) on university-owned devices	Low	Managed	3	Does not reach personal devices; may degrade calls
6	C6	Flow telemetry at the wireless edge [21], [22] to expose client-to-client flows and non-HTTP UDP flows	Medium	Network	3	Detects and does not prevent; sampling may miss short exchanges; needs privacy review
7	C7	MAC-spoofing safeguards: reject the gateway's MAC from wireless clients; drop a MAC seen on two networks	Medium	Network	2	Described as rarely available in [10]
8	C8	Per-client group keys, disabling shared group-addressed forwarding	Medium	Network	1	Affects services that rely on group-addressed traffic; needs access-point support
9	C9	Per-user isolation domain (one VLAN or equivalent per client)	High	Network	7	Implementation challenges noted in [10]; scale untested
10	C10	Addressing change on client networks: translation or short-lived addresses, for IPv4 and IPv6 alike	High	Network	3	Moves the leak from a public to a private address; no effect if IPv6 stays global (Section VIII); redesign of the address plan
11	C11	Egress restriction on outbound UDP to known real-time services	High	Network	1	Breaks legitimate applications; a channel can move to an allowed port or relay

eight repetitions give 560 sessions. All completed without a harness error. An earlier pilot of 140 sessions without the added delay is kept with the data; where it differs, the text says so.

C. Results

Table VI gives the path each session took and Table VII what the offer disclosed. Outcomes were the same in all eight sessions of a cell except in two cells, which are named below. For a cell with eight identical outcomes the exact 95% lower confidence bound on the underlying rate is 63%; for 16 it is 79% and for 56 it is 94%.

Isolation alone leaves a path; the gateway rule closes it (Q1). In configuration N3, isolation removed the link-layer path, and the raw-host browsers connected to each other through the gateway in 16 of 16 sessions. No attack technique was involved; this is ordinary WebRTC behaviour on a segment where the gateway forwards between clients. In N4, with the gateway rule added, none of the 56 sessions used a client-to-client path.

Multicast filtering moves default browsers to a relay, on IPv4 behind NAT only (Q2). On the open segment N1, default Chromium connected directly in all sessions, with or without microphone permission. With the mDNS filter as the only change (N2), all 16 of those sessions used the relay. The raw-host browsers were unaffected and connected directly in 16 of 16 sessions of N2. Firefox was not uniform in N1: its default configuration connected directly in 3 of 8 sessions and with microphone permission in 7 of 8, the rest using the relay. In three captured sessions without added delay, one of the two Firefox mDNS queries was answered on its retry about 3 s later, after the relayed pair had already connected. In the pilot without added delay, default Firefox used the relay in 5 of 5 sessions. WebKit used the relay in all sessions of N1. In N2 no session of default Firefox, Firefox with microphone permission, or WebKit was direct. The filter stopped having any effect once the segment carried global IPv6: in N9 all 56 sessions connected directly over IPv6.

Microphone permission did not change what was advertised. In all 160 sessions with microphone permission, Chromium and Firefox still advertised mDNS names and no raw host address.

The addressing plan decides what a default browser discloses (Q3). Under NAT with IPv4 only, the five configurations that keep obfuscation on never placed the interface address in the offer (0 of 200 sessions). The raw-host browsers always did (80 of 80), whatever controls were active. Under public IPv4 addressing every session disclosed it (112 of 112). For the browsers with obfuscation it travelled as the server-reflexive candidate, which on a segment without translation is the interface address itself. Under dual-stack addressing the private IPv4 address was protected exactly as under NAT, and the

TABLE VI
EMULATED SESSIONS: PATH TAKEN, BY NETWORK CONFIGURATION AND BROWSER CONFIGURATION. EIGHT SESSIONS PER CELL. D: DIRECT, G: VIA GATEWAY, R: RELAY, –: NO CONNECTION

	Network configuration					Path taken (count of sessions)						
Cfg	**Addressing**	**Isolation**	**mDNS**	**Gateway**	**Relay**	**Chromium**	**Chromium raw-host**	**Firefox**	**Firefox raw-host**	**WebKit**	**Chromium with mic.**	**Firefox with mic.**
N1	NAT	off	allowed	drops	yes	D 8	D 8	D 3 R 5	D 8	R 8	D 8	D 7 R 1
N2	NAT	off	filtered	drops	yes	R 8	D 8	R 8	D 8	R 8	R 8	R 8
N3	NAT	on	allowed	forwards	yes	R 8	G 8	R 8	G 8	R 8	R 8	R 8
N4	NAT	on	allowed	drops	yes	R 8	– 8	R 8	– 8	R 8	R 8	R 8
N5	NAT	on	allowed	drops	no	– 8	– 8	– 8	– 8	– 8	– 8	– 8
N6	public	off	filtered	drops	yes	D 8	D 8	D 8	D 8	D 8	D 8	D 8
N7	public	on	allowed	drops	yes	R 8	R 8	R 8	R 8	R 8	R 8	R 8
N8	NAT + IPv6	off	allowed	drops	yes	D 8	D 8	D 8	D 8	D 8	D 8	D 8
N9	NAT + IPv6	off	filtered	drops	yes	D 8	D 8	D 8	D 8	D 8	D 8	D 8
N10	NAT + IPv6	on	allowed	drops	yes	R 8	R 8	R 8	– 8	R 8	R 8	R 8

TABLE VII
SESSIONS IN WHICH THE OFFER CARRIED CLIENT 1'S OWN INTERFACE ADDRESS, BY ADDRESSING MODE AND ADDRESS FAMILY

Address	**Chromium**	**Chromium raw-host**	**Firefox**	**Firefox raw-host**	**WebKit**	**Chromium with mic.**	**Firefox with mic.**
NAT: private IPv4	0/40	40/40	0/40	40/40	0/40	0/40	0/40
Public IPv4	16/16	16/16	16/16	16/16	16/16	16/16	16/16
Dual: private IPv4	0/24	24/24	0/24	24/24	0/24	0/24	0/24
Dual: global IPv6	24/24	24/24	24/24	24/24	24/24	24/24	24/24

global IPv6 address was disclosed in 168 of 168 sessions, again as a server-reflexive candidate. Translating IPv4 while routing IPv6 globally therefore hid nothing that identifies the device. This agrees with the observation in [7] and extends it to obfuscating browsers and to the dual-stack case.

Sessions survive the controls only with a relay, and not always then (Q4). With isolation and the gateway rule on and no relay (N5), none of 56 sessions connected. With a relay, all 56 connected under public addressing (N7), 48 of 56 under dual-stack (N10), and 40 of 56 under NAT (N4), always through the relay. The failures are the raw-host configurations: both browsers in N4 and Firefox in N10. They reached the *failed* state after about 15 s and recurred in ten further sessions with a 60 s limit. Chromium raw-host connected through the relay in the same configuration when the added delay was removed (8 of 8 sessions: five in the pilot and three in a re-run). The failure therefore depends on timing between the candidate pairs and the translating gateway. We did not determine its mechanism, and it may be specific to the Linux translation used here.

D. Check on a Physical Wi-Fi Interface

The disclosure result (Q3) depends only on the browser and the addressing of the interface, so it can be checked on real hardware without a second device. A laptop (Windows 11, Intel Wi-Fi 7 BE201 adapter) ran Microsoft Edge 155 headless through Playwright on its everyday Wi-Fi connection, which gave it a private IPv4 address behind NAT and only a link-local IPv6 address. Each session started a fresh browser, gathered candidates against a public STUN server, and connected two peer connections inside the page. Edge ran in its default and raw-host configurations, 30 sessions each. Only address classes were stored.

In all 30 default sessions the offer carried one host candidate with an mDNS name and one server-reflexive candidate with a public IPv4 address different from the interface address; the private address never appeared. In all 30 raw-host sessions the host candidate was the private interface address, and the same server-reflexive candidate was present. No session produced an IPv6 candidate, because the interface had no global IPv6 address. All 60 loopback connections succeeded over a host pair (median connection time 109–110 ms). This matches the NAT rows of Table VII on a physical interface with a current browser build; it does not test isolation or a second device.

E. Check on an iPhone

An iPhone 16 Pro Max (iOS 26.0) shared its 5G connection through Personal Hotspot; a laptop on the hotspot served the released test page, and Safari on the iPhone gathered candidates against the same public STUN server. The page stores only address classes.

Both completed sessions gave the same result: two host candidates, each an mDNS name, and two server-reflexive candidates, one a public IPv4 address and one a global IPv6 address. No raw local address appeared. A third, two-device session was not completed and is not counted. Safari thus hid the local addresses but still gave the STUN server, and through it any peer, a public IPv4 and a global IPv6 address: the pattern of the dual-stack rows of Table VII. Because only classes were stored, we cannot say whether the IPv6 address was the device's own. Two sessions on one device show its behaviour; they do not measure a rate.

F. Published Mobile Evidence

The following is published evidence, not our measurement. For Android, the 2025 study reports that Chrome disclosed a LAN or carrier-grade NAT address and that Firefox, which protects the address on desktop, disclosed internal addresses on Android [6]; this is the behaviour the raw-host configuration stands for. For iOS, a 2021 test of Safari on iOS 15 observed two mDNS-obfuscated host candidates and one server-reflexive candidate [23]. With iCloud Private Relay switched on, the server-reflexive candidate still carried the device's real public address until iOS 15.1. A WebKit bug report from 2021, fixed in December 2022, describes Safari discarding the server-reflexive candidate when the host address is public and obfuscated [24], which bears on the public-addressing configurations N6 to N8. None of these sources covers IPv6 on a campus segment or is a systematic measurement of current iOS; the iPhone sessions above agree with the 2021 observation [23].

G. What the Results Change

The results support three statements in the framework, sharpen one, and qualify one. They support ranking the gateway rule C1 above isolation C3: isolation by itself left a working client-to-client path for raw-host browsers. They support the statement that network controls reduce the use of a disclosed address and not the disclosure: no control changed the 80 of 80 figure for raw-host browsers under NAT. They support treating the addressing plan (C10) as the one network-side measure that changes what is disclosed. They sharpen C10, which has to cover IPv6. A segment that translated IPv4 but handed out global IPv6 addresses disclosed a device address in every session, and gave every browser a direct path that the multicast filter did not touch. They also qualify C1 and C3. Together these make every same-segment session depend on a relay, so a web application that offers none stops working between two users on the same campus segment; in one addressing mode some sessions failed even with a relay.

The device checks support the disclosure part on real hardware. Not tested: any isolation-bypass attack from [10], [12], countermeasures C2 and C4 to C9 and C11, client isolation on a Wi-Fi access point, and Android browsers.

IX. Discussion

The role of WebRTC is narrower than the headline suggests. In two of the three trees the damaging steps belong to the isolation-bypass literature, and WebRTC supplies target selection. In the hostel tree and in step X1 it is the main mechanism. A campus operator should therefore not treat "WebRTC leakage" as one problem with one fix. Address disclosure is mostly a privacy and targeting issue that the network can make less useful but cannot stop; the data channel is a monitoring blind spot.

Network measures reduce the value of a leak, not the leak. No measure in Table V that the operator can enforce on personal devices stops a browser from disclosing an address. C1, C2, C7, and C9 make a disclosed local address harder to use. Only C10 changes what is disclosed, and it does so by substituting a less sensitive address.

Multicast filtering is a partial control that operators may already have. The mDNS draft warns that name resolution may fail where multicast is restricted [17]. In the emulation this happened: default sessions moved to the relay when mDNS was filtered. The effect does not extend to browsers that advertise raw addresses, which [6] reports for Android, and it disappeared when the segment had global IPv6. It cannot be counted as a countermeasure.

IPv6 deserves the operator's attention first. Address translation was the one network property that kept a default browser's address from the peer, and it applies to IPv4 only. A campus that runs dual-stack with global IPv6 on client segments is, for this leakage, in the position of the publicly addressed networks of [7], whatever its IPv4 plan.

Ethics. The paper describes attacks that are already public and adds no technique. The emulation ran on an isolated network on one computer. The device checks ran on the authors' own devices, contacted only a public STUN server and a test page on the authors' laptop, and stored address classes rather than addresses. No personal or institutional data was collected.

X. Limitations

1) *No live-network measurement.* The attack steps are taken from cited sources or marked as assumptions, and none was reproduced. The evaluation covers ordinary browser behaviour under four controls, not the isolation-bypass attacks.
2) *Emulation, not Wi-Fi.* The client segment is a Linux bridge. Bridge port isolation stands in for Wi-Fi client isolation and does not reproduce any vendor's implementation, its weaknesses, or radio effects. The physical-interface check covers one laptop, one browser, and one IPv4-only network behind NAT, and tests disclosure only.
3) *Browsers.* Only headless desktop Linux builds of Chromium, Firefox, and WebKit were used. The raw-host configuration is a switch on those builds, not an Android or iOS browser, and the WebKit build is not Safari. Mobile evidence is two sessions on one iPhone (iOS 26.0), on a cellular connection rather than campus Wi-Fi, and published sources in Section VIII-F, two of them informal (a blog post and a bug report). Microphone permission was granted through a fake device. Sessions used one STUN and TURN implementation, one added delay, and eight repetitions per cell, so rates between about 60% and 100% cannot be told apart within a cell. The failure of raw-host sessions under translation is reported and not explained.
4) *Reference model, not a real network.* The architecture combines RFC 7593 with one published description of a university deployment. Assumptions A1 to A4 may be false at a given institution, and the trees change if they are.

5) *Judgement-based ratings.* Burden and enforcement are ordinal ratings assigned by the authors. No cost data, vendor survey, or expert elicitation supports them, and coverage counts steps without weighting them by likelihood or impact. A different ordering rule, for example coverage first, would move C9 to the top.
6) *Evidence quality and search.* The main leakage source is an unreviewed preprint and the covert-channel evidence is one vendor report (Section V). The literature search used one index and was screened by two of the authors without a formal protocol.
7) *Scope.* Attacks on authentication, rogue access points, and wired residence networks are not modelled.

XI. Conclusion and Future Work

This paper joined work on WebRTC address leakage, Wi-Fi client-isolation bypass, and eduroam into a threat model for a university BYOD wireless network, with twenty attack steps in three campus scenarios and eleven ranked countermeasures. Under a rule that puts deployment burden first, gateway filtering and VLAN separation rank highest, per-user isolation is the strongest measure and the most expensive, and client isolation is a baseline that must not be relied on alone. An emulation with 560 browser sessions, a 60-session check on a physical Wi-Fi interface, and two sessions on an iPhone, agreed with the ranking on the points it could test and added two findings. Isolation alone left a path through the gateway for browsers that advertise raw addresses, and the gateway rule closed it. Address translation, not mDNS obfuscation or multicast filtering, decided whether a default browser's address reached the peer, and global IPv6 on the client segment undid that protection in every session. The iPhone showed the same pattern on a cellular connection.

Future work should repeat the evaluation on Wi-Fi access points and with Android and more iOS devices, reproduce the measurements on a campus network with the operator's approval, check assumptions A1 to A4, explain the relay failures under translation, and replace the ordinal ratings with administrators' estimates.

Data Availability

The testbed and analysis scripts, the raw session records, the device-check data, and the paper source are available at https://github.com/shubhisingh1510/cnpaperwork.

Acknowledgment

In line with IEEE policy on AI-generated content, the authors disclose that Claude (Anthropic), an AI assistant, was used to help draft and edit the text, to write the testbed, analysis, and test-page scripts, to execute the emulated evaluation and the laptop measurement on the authors' computer, and to format the manuscript. The iPhone measurement was carried out by the authors. The authors reviewed the content and take full responsibility for it.

References

[1] A. Keranen, C. Holmberg, and J. Rosenberg, "Interactive connectivity establishment (ICE): A protocol for network address translator (NAT) traversal," IETF RFC 8445, Jul. 2018.
[2] J. Uberti and G. Shieh, "WebRTC IP address handling requirements," IETF RFC 8828, Jan. 2021.
[3] E. Rescorla, "Security considerations for WebRTC," IETF RFC 8826, Jan. 2021.
[4] N. M. Al-Fannah, "One leak will sink a ship: WebRTC IP address leaks," arXiv:1709.05395, Sep. 2017.
[5] A. Fakis, G. Karopoulos, and G. Kambourakis, "Neither denied nor exposed: Fixing WebRTC privacy leaks," *Future Internet*, vol. 12, no. 5, p. 92, May 2020.
[6] A. F. K. Koysha, A. Boyaci, and R. Akdeniz, "WebRTC metadata and IP leakage in modern browsers: A cross-platform measurement study," arXiv:2510.16168, Oct. 2025.
[7] G. Nibert, S. Tixeuil, B. Polvé, N. J. Bakalafoua M'boussi, and X. S. Nguyen, "Preventing WebRTC IP address leaks," in *Risks and Security of Internet and Systems (CRiSIS 2024)*, ser. Lecture Notes in Computer Science, vol. 15456. Cham, Switzerland: Springer, 2025, pp. 365–381.
[8] K. Wierenga, S. Winter, and T. Wolniewicz, "The eduroam architecture for network roaming," IETF RFC 7593, Sep. 2015.
[9] J. H. Cox, R. Clark, and H. Owen, "Leveraging SDN and WebRTC for rogue access point security," *IEEE Trans. Netw. Service Manag.*, vol. 14, no. 3, pp. 756–770, Sep. 2017.
[10] X. Zhou, J. Pu, Z. Liu, Z. Qian, Z. Tan, S. V. Krishnamurthy, and M. Vanhoef, "AirSnitch: Demystifying and breaking client isolation in Wi-Fi networks," in *Proc. Network and Distributed System Security Symp. (NDSS)*, San Diego, CA, USA, Feb. 2026.
[11] Sansec, "Novel WebRTC skimmer bypasses security controls at $100+ billion car maker," Mar. 2026, [Online]. Available: https://sansec.io/research/webrtc-skimmer.
[12] D. Schepers, A. Ranganathan, and M. Vanhoef, "Framing frames: Bypassing Wi-Fi encryption by manipulating transmit queues," in *Proc. 32nd USENIX Security Symp.*, Anaheim, CA, USA, Aug. 2023.
[13] M. Vanhoef, "MacStealer: Wi-Fi client isolation bypass (CVE-2022-47522)," 2023, [Online]. Available: https://github.com/vanhoefm/macstealer.
[14] E. T. Tchao, R. Y. Ansah, and S. D. Djane, "Barrier free internet access: Evaluating the cyber security risk posed by the adoption of bring your own devices to e-learning network infrastructure," arXiv:1710.08795, Oct. 2017.
[15] M. Petit-Huguenin, G. Salgueiro, J. Rosenberg, D. Wing, R. Mahy, and P. Matthews, "Session traversal utilities for NAT (STUN)," IETF RFC 8489, Feb. 2020.
[16] J. Weil, V. Kuarsingh, C. Donley, C. Liljenstolpe, and M. Azinger, "IANA-reserved IPv4 prefix for shared address space," IETF RFC 6598, Apr. 2012.
[17] Y. Fablet, J. Uberti, J. de Borst, and Q. Wang, "Using multicast DNS to protect privacy when exposing ICE candidates," IETF Internet-Draft draft-ietf-mmusic-mdns-ice-candidates-03 (expired), Dec. 2021.
[18] T. Mirzoev and S. White, "The role of client isolation in protecting Wi-Fi users from ARP spoofing attacks," *i-manager's J. Inf. Technol.*, vol. 1, no. 2, 2012, arXiv:1404.2172.
[19] S. Tang, E. Alowaisheq, X. Mi, Y. Chen, X. Wang, and Y. Dou, "Stealthy peers: Understanding security risks of WebRTC-based peer-assisted video streaming," arXiv:2212.02740, Dec. 2022.
[20] B. Schneier, "Attack trees," *Dr. Dobb's J.*, vol. 24, no. 12, pp. 21–29, Dec. 1999.
[21] P. Phaal, S. Panchen, and N. McKee, "InMon Corporation's sFlow: A method for monitoring traffic in switched and routed networks," IETF RFC 3176, Sep. 2001.
[22] T. Dietz, B. Claise, P. Aitken, F. Dressler, and G. Carle, "Information model for packet sampling exports," IETF RFC 5477, Mar. 2009.
[23] P. Hancke, "Apple's not-so-private relay fails with WebRTC," webrtcHacks, Sep. 2021, [Online]. Available: https://webrtchacks.com/apples-not-so-private-relay-fails-with-webrtc/.
[24] K. Lee, "A server-reflexive candidate is discarded when the host IPv4 address is public and obfuscated," WebKit Bugzilla, bug 233414, Nov. 2021, [Online]. Available: https://bugs.webkit.org/show_bug.cgi?id=233414.